DEVOPS HANDS-ON PROJECT · FULL TECHNICAL DOCUMENTATION

Jenkins Controller–Agent Docker Deployment on AWS EC2

Complete implementation guide, real troubleshooting, commands and screenshots

Ubuntu · Jenkins · RSA SSH · Docker · Nginx

Project documentation assembled from recorded work on 23 October 2026.

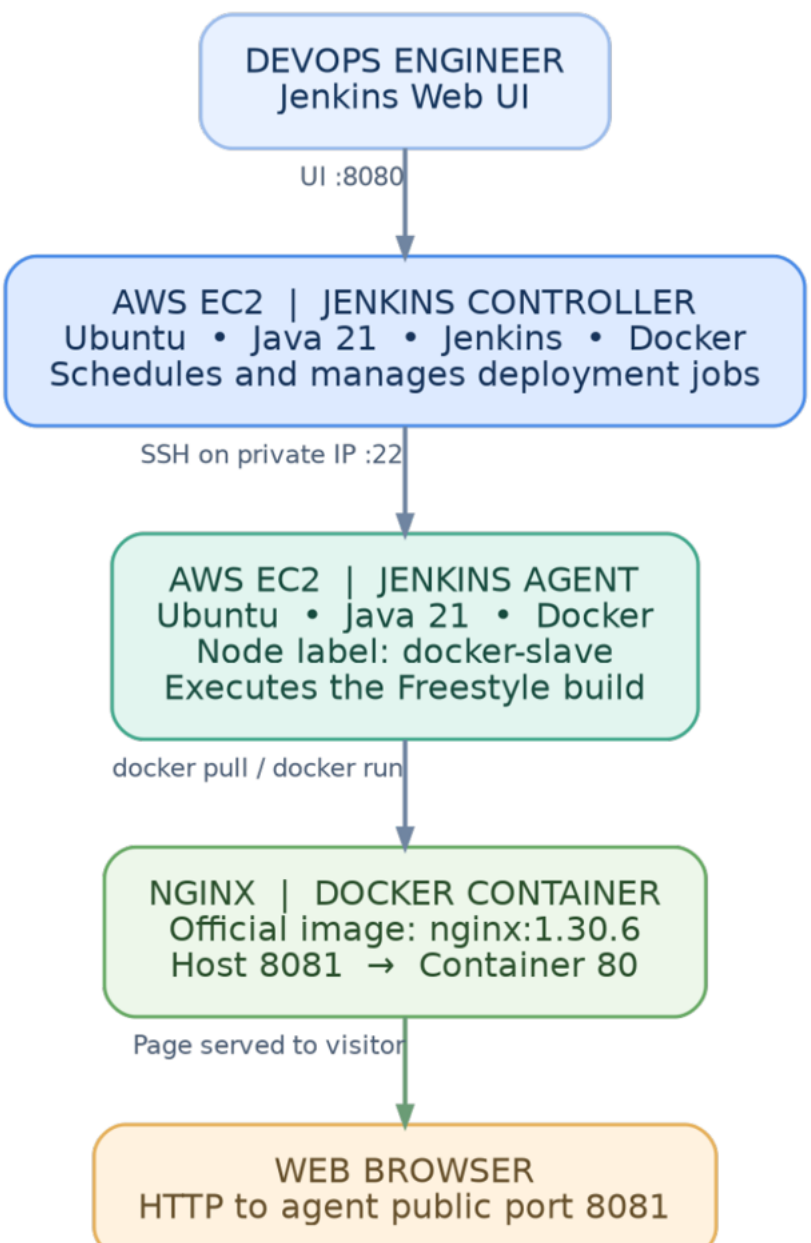
Scope: Freestyle jobs triggered through Jenkins; Docker containers executed on the remote agent. GitHub webhooks and declarative pipelines are future improvements, not completed work.

SECTION 01

Jenkins Controller–Agent Docker Deployment on AWS EC2

Hands-on DevOps case study | Jenkins + Docker + Nginx + AWS EC2 + Linux + SSH

> **Outcome:** Created a Jenkins Freestyle deployment job on an AWS EC2 **controller** and executed it on a separate EC2 **agent**, where Docker ran an official Nginx container accessible through the agent's public endpoint.
>
> This documents a completed lab. It is **remote deployment automation**, not yet a full Git-triggered CI/CD pipeline.



At a glance

Item	Lab implementation
Cloud	AWS EC2, two Ubuntu Linux instances
Controller	Jenkins LTS installed as a Linux service, with Java 21 and Docker Engine
Agent	Java 21, Docker Engine, SSH server, Jenkins agent launched via SSH
SSH	RSA 4096-bit key pair; controller connects to agent's **private** IP on TCP/22
Scheduling	Jenkins permanent node `docker-slave`, 1 executor, label `docker-slave`
Jenkins job	Freestyle `nginx-docker-deployment` restricted to `docker-slave`
Container image	`nginx:1.30.6` — Docker Official Image
Ports	Jenkins UI `8080` on controller; Nginx host `8081` → container `80` on agent
Result	Remote agent job **SUCCESS**; Docker access tested; Nginx reached in a browser

How it works

1. An engineer creates or triggers a Jenkins job from the controller UI.
2. The controller selects the node labeled `docker-slave` and communicates with it over SSH.
3. The agent process executes the build steps as the Linux user `ubuntu`.
4. Docker Engine **on the agent** pulls and starts the official Nginx image.
5. The application is reached at `http://<AGENT_PUBLIC_IP>:8081`, with its container listening on TCP/80.

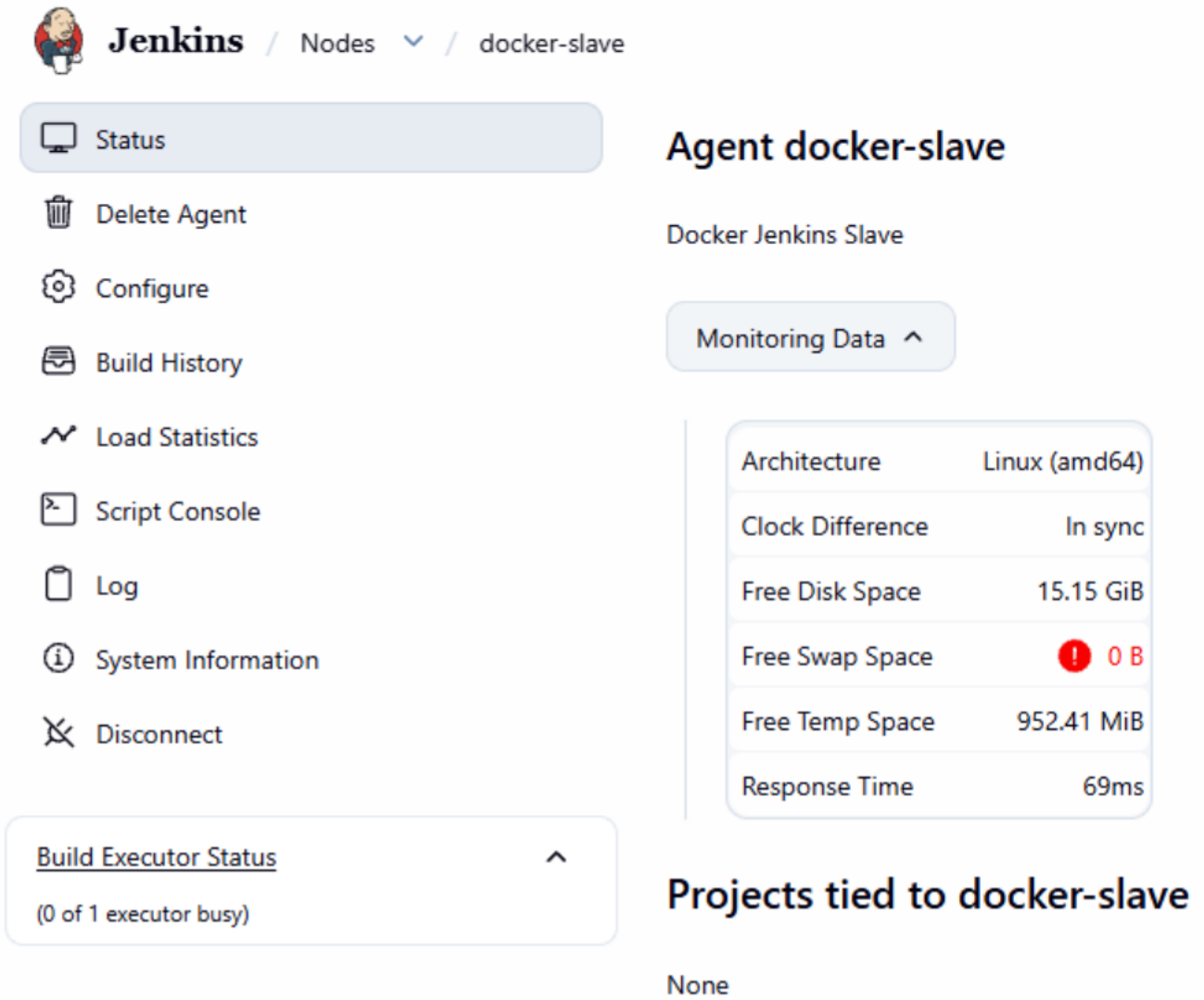
The controller manages and schedules work; it does **not** host the deployed Nginx container. Docker was installed on the controller as part of the lab, but these jobs ran Docker on the **agent**.

Reproduction path

Step	Action	Guide
1	Launch two EC2 instances, apply least-privilege security group rules	Infrastructure and tool installation
2	Install Java, Jenkins, Docker on controller; Java, Docker on agent	Infrastructure and tool installation
3	Generate an RSA key for the Jenkins service user, authorize its public key on the agent	SSH connection and agent
4	Add private-key credentials, register `docker-slave`, and bring node online	SSH connection and agent
5	Grant the dedicated training agent's `ubuntu` user Docker access and reconnect agent	Job setup and deployment
6	Prove remote execution, then deploy Nginx using a Freestyle job	Job setup and deployment
7	Verify container status, HTTP response and browser access	Validation and cleanup

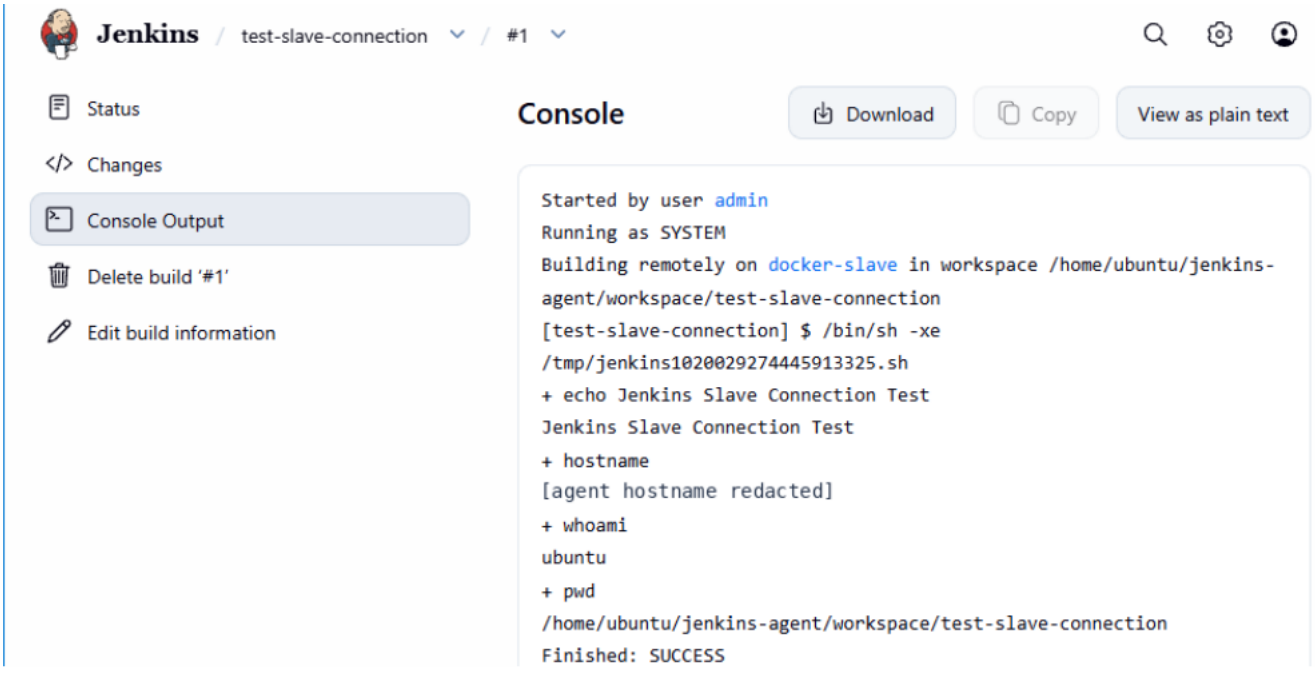
Tested milestones

1. Agent connected and online



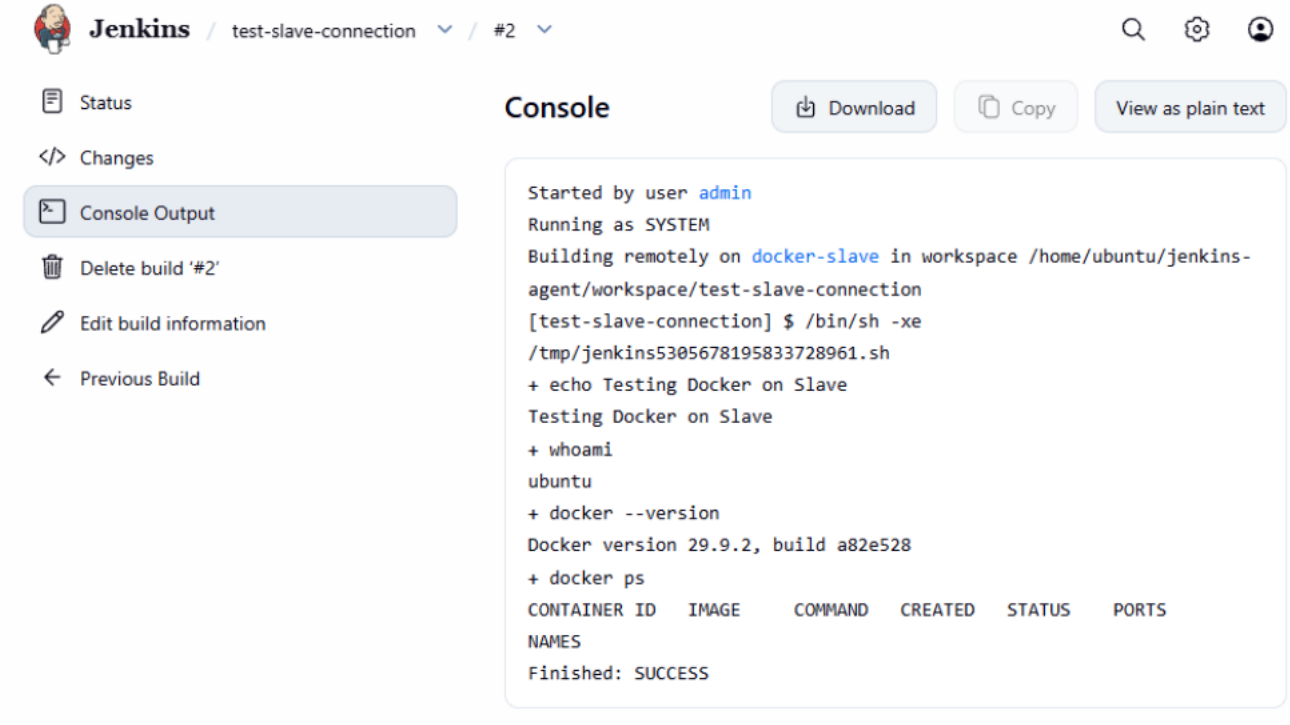
The dashboard showed agent monitoring details and **Disconnect** / **Mark this node temporarily offline** controls; one executor was available.

2. Controller executed shell commands on the agent



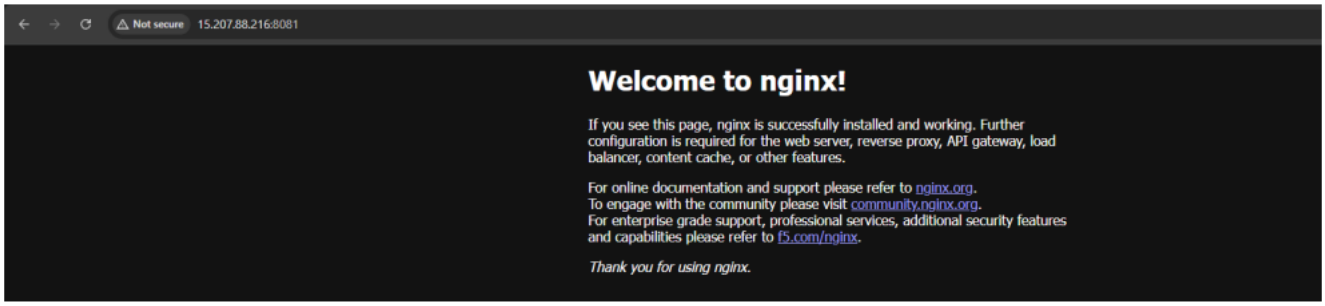
Console evidence included **Building remotely on `docker-slave`**, `whoami` → `ubuntu`, a remote workspace under `/home/ubuntu/jenkins-agent/workspace/`, and **Finished: SUCCESS**.

3. Agent's Docker daemon was available to Jenkins



In the second test build, the controller executed `docker --version` and `docker ps` on the agent as `ubuntu`; the job finished successfully.

4. Nginx served traffic



Final Nginx website output: welcome page successfully displayed on port 8081.

Troubleshooting that happened in this lab

Failure	Root cause	Fix
`Identity file .../jenkins_slave not accessible`	SSH command referenced an obsolete filename instead of the generated `jenkins_slave_rsa`	Used the correct key path
`Permission denied (publickey)`	Agent's authorized key was a different RSA key (2048-bit, nonmatching SHA256 fingerprint); controller held RSA 4096-bit key	Added the matching controller RSA **public** key on the agent; retained existing key
Agent offline due to temporary disk warning	`/tmp` was a nearly empty 953 MiB **tmpfs**, smaller than Jenkins' default 1 GiB *free-space threshold*	Lowered the agent's Free Temp Space Threshold (lab configuration) and brought it online
`Free Swap Space: 0 B` warning	No swap configured	Observed as a resource warning; not the cause of the original offline state

Full commands and explanations: Troubleshooting case study.

Security notes

- **Do not commit private keys, Jenkins credential exports, passwords, AWS access keys, or full infrastructure identifiers.** The included screenshots are sanitized copies of actual session screenshots.
- Limit Jenkins HTTP `8080`, SSH access and public Nginx `8081` to approved source networks; use HTTPS/reverse proxy for real deployments.
- Grant agent TCP/22 inbound from the controller's **security group** and ensure matching controller egress if egress is restricted.
- Adding `ubuntu` to the `docker` group grants **root-equivalent host control**. This is acceptable only for an isolated learning agent; production designs should use tightly isolated build workers and explicit authorization.

- `docker run -p 8081:80` publishes on all host interfaces by default; the EC2 Security Group is a key protection, not an optional step.
- The key used here had **no passphrase** for unattended SSH automation, which requires extra care protecting the Jenkins Credentials store.
- Docker image tag `1.30.6` names a version, but a registry tag can be republished; for exact bit-for-bit reproducibility, pin a verified `sha256` image digest.

Read Publishing checklist before uploading anything from your own environment.

Project layout

```
jenkins-docker-aws-deployment/
├── README.md
├── docs/
│   ├── 01-infrastructure-setup.md
│   ├── 02-ssh-and-agent.md
│   ├── 03-jenkins-jobs-and-deployment.md
│   ├── 04-troubleshooting.md
│   ├── 05-validation-and-cleanup.md
│   └── 06-official-references.md
├── scripts/
│   ├── deploy-nginx.sh
│   └── verify-agent.sh
├── assets/
│   ├── diagrams/architecture.png
│   ├── diagrams/architecture.svg
│   └── screenshots/ (sanitized actual evidence)
```

```
├── LINKEDIN_POST.md
├── Jenkins_Docker_AWS_Project_Case_Study.pdf
├── PUBLISHING_CHECKLIST.md
└── .gitignore
```

Future improvements (not implemented in this lab)

- Store a `Jenkinsfile` in Git; replace Freestyle with a declarative pipeline.
- Integrate GitHub and webhook triggers after validating credentials and SCM security.
- Add tests, image scanning, image digest pinning, release strategy and rollback.
- Deploy application-specific images rather than a prebuilt Nginx example.
- Use HTTPS and dedicated, short-lived, least-privilege agents for production.

Sources

Every software installation and primary command is linked in Official references. All evidence screenshots are from this project and sanitized before publication.

Scope note: This repository is a reproducible lab case study, not a production-ready CI/CD system. No AWS resources or Jenkins services are created by cloning this repository.

SECTION 02

01 — Infrastructure and Tool Installation

Architecture prerequisites

Two Ubuntu EC2 instances with private IP connectivity, usually in one VPC:

Host	Role	Installed components
`master`	Jenkins **controller**	Ubuntu Linux, Java 21, Jenkins LTS service, Docker Engine
`slave`	Jenkins **agent**	Ubuntu Linux, Java 21, Docker Engine, SSH server; **no full Jenkins server installation**

The hostnames are explanatory labels; the agent's actual Jenkins node name is `docker-slave`. Instance types, Ubuntu release, and exact public IPs are intentionally omitted (not recorded for this project).

Suggested EC2 security groups

Instance	Direction	TCP port	Narrowest practical source/destination
Controller	Inbound	22	Your admin public IP /32 (only if direct SSH administration is needed)
Controller	Inbound	8080	Your admin public IP /32 (lab Jenkins UI; not the open internet)
Controller	Outbound	22	Agent SG, if outbound is not already permitted
Agent	Inbound	22	**Controller security group ID**; plus your admin IP if separately needed
Agent	Inbound	8081	Your admin public IP /32 for browser validation
Both	Outbound	443 / DNS	Only destinations required for package/image downloads, subject to your network rules

AWS Security Groups are stateful. Referencing the controller security group for agent TCP/22 allows private traffic from controllers in that group when the topology supports security-group referencing.

Do not require inbound TCP/2375 or 2376: Docker here is accessed locally by the agent user; no remote Docker API is exposed.

Controller: Java + Jenkins

Commands used for Ubuntu APT-based installation follow the official Jenkins Linux guide.

```
sudo apt update
sudo apt install -y fontconfig openjdk-21-jre wget
java -version

sudo install -m 0755 -d /etc/apt/keyrings
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
  https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key

echo 'deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable
binary/' | \
  sudo tee /etc/apt/sources.list.d/jenkins.list

sudo apt update
```

```
sudo apt install -y jenkins
sudo systemctl enable --now jenkins
sudo systemctl status jenkins
```

Open `http://<CONTROLLER_PUBLIC_IP>:8080` from your approved admin IP and unlock Jenkins. The initial password **must not be published**; retrieve it locally with:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

Install suggested plugins and complete the administrator setup. The **SSH Build Agents** plugin (`ssh-slaves` plugin ID) was already installed/enabled in this lab.

Controller and agent: Docker Engine

These commands reproduce the Docker **official Ubuntu APT repository** method (Docker documentation). Run on **both** instances.

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
sudo systemctl status docker
sudo docker run --rm hello-world
```

> **Before installation:** Docker officially recommends checking/removing **conflicting distro Docker packages** (`docker.io`, `podman-docker`, etc.) if present. Do not blindly remove packages on a populated server. See official prerequisites.

On the **agent**, also ensure Java 21 is installed:

```
sudo apt update
sudo apt install -y fontconfig openjdk-21-jre
java -version
```

Checks:

```
docker --version
sudo docker ps
```

Observed in the lab's agent job log: `Docker version 29.9.2`. Ubuntu patch version and Java patch release were not captured.

Why Docker on the controller?

It was installed to meet this lab's stated setup requirement. **The demonstrated deployment does not use the controller's Docker daemon.** The Freestyle job is restricted to the agent and invokes the agent's local Docker daemon.

SECTION 03

02 — SSH Keys, Jenkins Credentials and Agent Registration

A. Generate an RSA key *as the Jenkins service user* on the controller

The completed lab used a **4096-bit RSA** key pair and the filename `jenkins_slave_rsa`:

```
sudo -u jenkins mkdir -p /var/lib/jenkins/.ssh
sudo chown jenkins:jenkins /var/lib/jenkins/.ssh
sudo chmod 700 /var/lib/jenkins/.ssh

sudo -u jenkins ssh-keygen -t rsa -b 4096 \
  -f /var/lib/jenkins/.ssh/jenkins_slave_rsa -N ''

sudo ls -l /var/lib/jenkins/.ssh/
```

- Private key: `/var/lib/jenkins/.ssh/jenkins_slave_rsa` — **never commit or send**.
- Public key: `/var/lib/jenkins/.ssh/jenkins_slave_rsa.pub` — copy to the agent.
- `-t rsa`: RSA algorithm; `-b 4096`: 4096-bit modulus; `-f`: path; `-N ''`: no passphrase (lab automation only).

For a production environment, prefer an organization-approved key type, credential policies and secret lifecycle. Do not assume RSA is inherently superior to Ed25519.

B. Authorize the public key on the agent

On the controller, display **only** the public key:

```
sudo cat /var/lib/jenkins/.ssh/jenkins_slave_rsa.pub
```

On the agent, log in as `ubuntu` (using your existing EC2 admin access):

```
mkdir -p /home/ubuntu/.ssh
chmod 700 /home/ubuntu/.ssh
nano /home/ubuntu/.ssh/authorized_keys
# Paste controller's *public* key on its own line; keep existing login key(s).
chmod 600 /home/ubuntu/.ssh/authorized_keys
sudo chown -R ubuntu:ubuntu /home/ubuntu/.ssh
```

Correctly troubleshoot key mismatches by fingerprint, not by key size alone:

```
# Controller
sudo ssh-keygen -lf /var/lib/jenkins/.ssh/jenkins_slave_rsa.pub

# Agent
ssh-keygen -lf /home/ubuntu/.ssh/authorized_keys
```

A matching fingerprint must appear among the agent's authorized keys. The project originally had a different RSA 2048-bit authorized key; it was **not a converted** version of the new RSA 4096-bit key.

C. Connect from controller through the private network

Make sure agent SG allows TCP/22 inbound from controller SG; check controller TCP/22 egress if restricted.

```
sudo -u jenkins ssh -o IdentitiesOnly=yes \
  -i /var/lib/jenkins/.ssh/jenkins_slave_rsa \
  ubuntu@<AGENT_PRIVATE_IP>
```

Verify the *agent SSH host key fingerprint* before trusting the first connection; this is different from the user's RSA **authentication** key fingerprint. A verified first connection typically writes the host key to the Jenkins user's `known_hosts` file. Use `exit` to leave the agent.

D. Create Jenkins SSH credential

Jenkins Dashboard → **Manage Jenkins → Credentials → System → Global credentials → Add Credentials**:

Field	Value
Kind	**SSH Username with private key**
Username	`ubuntu`
Private key	**Enter directly**; paste controller's private key securely into Jenkins UI
Passphrase	Blank, because `-N ''` was used
ID	`slave-ssh-key`
Description	`Jenkins controller to agent SSH`

For this lab, the private key can be displayed locally via:

```
sudo cat /var/lib/jenkins/.ssh/jenkins_slave_rsa
```

E. Prepare the agent filesystem

On the agent, while logged in as `ubuntu`:

```
mkdir -p /home/ubuntu/jenkins-agent
java -version
```

F. Register agent in Jenkins

Jenkins Dashboard → **Manage Jenkins → Nodes → New Node** → Name `docker-slave` → **Permanent Agent**.

Setting	Value
Executors	1
Remote root	`/home/ubuntu/jenkins-agent`
Labels	`docker-slave`

Setting	Value
Usage	Only build jobs with label expressions matching this node
Launch method	Launch agents via SSH
Host	`<AGENT_PRIVATE_IP>` (**private**, not public)
Credentials	`slave-ssh-key`
Port	22
Host Key Verification Strategy	Known hosts file Verification Strategy
Availability	Keep this agent online as much as possible

Save and check **Nodes → docker-slave → Log**. The connection succeeded in this lab, and later console jobs proved it could run remote shell commands.

For dedicated-controller practice, set the built-in controller node's executor count to `0` so your application builds run only on agents. Be sure at least one eligible agent exists for jobs you run.

Read the official Jenkins agent guide and SSH Build Agents plugin.

SECTION 04

03 — Jenkins Freestyle Jobs and Docker Deployment

A. Configure Docker access on the agent

The SSH-launched agent executes as `ubuntu`; the Docker CLI needs access to `/var/run/docker.sock`.

On the agent:

```
getent group docker
sudo usermod -aG docker ubuntu
id ubuntu
```

Disconnect and relaunch the Jenkins node so the running SSH agent process receives updated supplementary groups. Test under the new agent connection:

```
docker --version
docker ps
```

Security warning: Docker group membership provides root-equivalent privileges. This is for the isolated lab only. See Docker post-install docs.

B. Test that jobs run on the agent

From Jenkins controller UI:

1. **New Item** → `test-slave-connection` → **Freestyle project**.
2. In General, check **Restrict where this project can be run**.
3. Label expression: `docker-slave`.
4. **Build Steps → Execute shell**: paste the following.
5. Save → **Build Now** → **Console Output**.

```
echo 'Jenkins Slave Connection Test'
hostname
whoami
pwd
```

The observed **#1** test build proved `Building remotely on docker-slave`, user `ubuntu`, agent workspace and `Finished: SUCCESS`.

Then replace the shell commands with the Docker access test:

```
echo 'Testing Docker on Slave'
whoami
docker --version
docker ps
```

The observed **#2** test build returned `Docker version 29.9.2`, an initially empty `docker ps` list and `Finished: SUCCESS`.

> Jenkins console text `Running as SYSTEM` describes its *internal security identity*; the Linux shell's `whoami` confirmed `ubuntu`.

C. Deploy official Nginx image

1. **New Item** → `nginx-docker-deployment` → **Freestyle project**.
2. **Restrict where this project can be run** → `docker-slave`.
3. Under **Build Steps**, select **Execute shell**.
4. Paste the contents of `../scripts/deploy-nginx.sh`, or its equivalent inline commands below.
5. Save → **Build Now** → **Console Output**.

```
set -e

echo 'Deploying Nginx on Jenkins Slave'
docker pull nginx:1.30.6

if docker container inspect jenkins-nginx >/dev/null 2>&1; then
    docker container rm --force jenkins-nginx
fi

docker run -d \
    --name jenkins-nginx \
    -p 8081:80 \
    nginx:1.30.6

docker ps
curl --fail --head http://127.0.0.1:8081/
```

The repository's `deploy-nginx.sh` uses the same logic plus **HTTP retries** to tolerate small startup delays. If entering the script in a Jenkins shell field, `set -e` and POSIX-compatible commands work with its `/bin/sh` launcher; a file executed via `bash scripts/deploy-nginx.sh` uses its Bash shebang.

Why `nginx:1.30.6`?

It is a **version-specific, official, stable-series Nginx tag** verified in the Docker Official Image manifest and Docker Hub. `nginx:stable` is also official, but moves as stable releases change. A fixed tag does **not** guarantee immutable bytes; use a verified SHA256 digest for immutable pinning.

Command reference

Command	Purpose
`docker pull`	Fetch the image from a registry
`docker container inspect`	Check whether a previous named container exists
`docker container rm --force`	Remove a previous container before recreation; causes brief downtime
`docker run -d --name ... -p 8081:80`	Start in background; map **agent host TCP/8081** to **container TCP/80**
`docker ps`	List running containers
`curl --fail --head http://127.0.0.1:8081/`	Verify HTTP headers locally, failing on HTTP errors

Port security: without a host IP specified, Docker publishes the mapped port on all interfaces by default. Restrict public reachability using the agent's AWS security group. The app is not served from the controller.

D. Verify in your browser

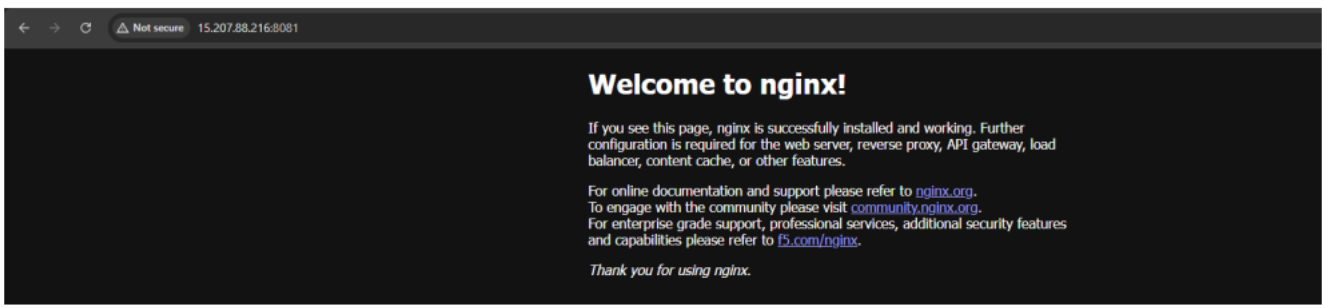
On agent:

```
docker ps --filter name=jenkins-nginx
curl -I http://127.0.0.1:8081/
```

Then, from your approved client:

```
http://<AGENT_PUBLIC_IP>:8081
```

Nginx browser output



The supplied browser screenshot shows the Nginx welcome page on port 8081.

SECTION 05

04 — Troubleshooting Case Study (Actual Incidents)

This section records **errors actually encountered**, along with relevant commands and the eventual working state. Values shown in commands use placeholders to protect the instance's private addressing.

Incident 1 — SSH private-key filename did not exist

Error

```
Warning: Identity file /var/lib/jenkins/.ssh/jenkins_slave not accessible: No such file or
directory.
Permission denied (publickey).
```

Cause — The generated RSA key had been named `jenkins_slave_rsa`, but the SSH test referred to an older suggested name `jenkins_slave`. The missing file prevented SSH from using the intended private key.

Fix — Verify the file and select the actual key:

```
sudo ls -l /var/lib/jenkins/.ssh/
sudo -u jenkins ssh -o IdentitiesOnly=yes \
  -i /var/lib/jenkins/.ssh/jenkins_slave_rsa \
  ubuntu@<AGENT_PRIVATE_IP>
```

The filename fix alone did **not** resolve the next issue.

Incident 2 — RSA public key did not match

Symptoms — SSH continued to return `Permission denied (publickey)`. Agent-side log also contained an EC2 Instance Connect `AuthorizedKeysCommand ... failed, status 22` message; that line by itself did **not** prove a local `authorized_keys` mismatch.

Evidence — The controller's new RSA public key measured **4096 bits**, while the public key found in agent `authorized_keys` measured **2048 bits**, and their **SHA256 fingerprints differed**.

Diagnosis

```
# Controller
sudo ssh-keygen -lf /var/lib/jenkins/.ssh/jenkins_slave_rsa.pub

# Agent
ssh-keygen -lf /home/ubuntu/.ssh/authorized_keys
```

Root cause — Two **distinct RSA key pairs**, not key size conversion. The correct controller-generated 4096-bit RSA public key was missing from the agent user's authorized keys.

Resolution — Copied the entire `.pub` key from controller to agent `/home/ubuntu/.ssh/authorized_keys` on its own line while retaining existing EC2 login keys. Applied ownership and permissions:

```
sudo chown -R ubuntu:ubuntu /home/ubuntu/.ssh
sudo chmod 700 /home/ubuntu/.ssh
sudo chmod 600 /home/ubuntu/.ssh/authorized_keys
```

Verification — Matching fingerprint appeared; controller successfully connected as `ubuntu` to the agent's **private** IP without a password.

> RSA 4096-bit keys **do not become RSA 2048-bit keys during SSH or copying**. `authorized_keys` can contain several independent authorized public keys.

Incident 3 — Jenkins agent marked offline for temp-space threshold

Warning in UI

```
Disk space is below threshold of 1.00 GiB.
Only 952.41 MiB ... left on /tmp.
```

Check on agent

```
df -hT /tmp
```

Observed values:

```
Filesystem  Type   Size   Used  Avail  Use%  Mounted on
tmpfs       tmpfs  953M   164K  953M   1%   /tmp
```

Root cause — `/tmp` was an almost-empty memory-backed `tmpfs` with less **total** free capacity than the Jenkins **default monitoring threshold of 1 GiB**. This was **not** a 99%-full disk and not an SSH issue.

Resolution — Under **Manage Jenkins → Nodes → docker-slave → Configure → Node Properties → Disk Space Monitoring Thresholds**, use a reduced **Free Temp Space Threshold** appropriate for an isolated lab (e.g. `256MiB`, with warning threshold `512MiB`), save, and click **Bring this node back online**. More storage/appropriate sizing is preferable in production.

Verification — Jenkins agent monitoring data became visible and the **Disconnect** action was available; subsequent Jenkins Freestyle jobs ran successfully.

Unrelated observation — `Free Swap Space: 0 B` was shown in red on the monitoring dashboard. It is not proof that swap exhaustion caused this temporary-space offline state.

Incident 4 — Docker daemon access for remote jobs

Requirement — The agent connects via SSH as `ubuntu`, but Docker's UNIX socket is not generally available to a standard user by default.

Change applied as part of lab setup

```
sudo usermod -aG docker ubuntu
id ubuntu
```

Then reconnect the Jenkins agent to refresh its supplementary groups.

Verification — Jenkins build #2 executed `docker --version` and `docker ps` without `sudo` and returned `Finished: SUCCESS`. **A `permission denied` Docker error was not captured** in the supplied console output, so we do not invent one.

Risk — Docker's official docs warn that `docker` group membership grants root-level privileges. Use a disposable isolated agent and avoid this arrangement on shared production hosts.

Evidence images

- Offline monitoring warning
- Temp space filesystem output
- Agent online
- Remote job succeeded
- Docker CLI worked from Jenkins

SECTION 06

05 — Validation, Evidence Collection and Lab Cleanup

What was actually verified

Check	Evidence or source	State
SSH controller → agent	Successful SSH connection described in project session	Completed
Agent online in Jenkins	Real sanitized node dashboard screenshot	Completed
Jenkins job ran on agent	Freestyle build #1 console `Building remotely on docker-slave`, `whoami` `ubuntu`	Completed
Jenkins could run Docker	Freestyle build #2 console Docker `29.9.2`, `docker ps`, SUCCESS	Completed
Nginx site accessible	Project owner successfully opened Nginx in browser	Completed (no webpage screenshot supplied)

Quick on-agent checks

```
docker ps --filter name=jenkins-nginx
docker container inspect jenkins-nginx --format '{{.State.Status}}'
curl --fail --head http://127.0.0.1:8081/
```

`curl` should return a successful HTTP status such as `200 OK` if the default Nginx page is available. A container may be running even if HTTP readiness is not yet achieved; check logs if health checks fail:

```
docker logs --tail 50 jenkins-nginx
```

SECTION 07

06 — Official Documentation and Sources

Verified for this write-up: 23 October 2026. Product versions and UI labels evolve; prefer these vendor-owned pages when repeating the lab.

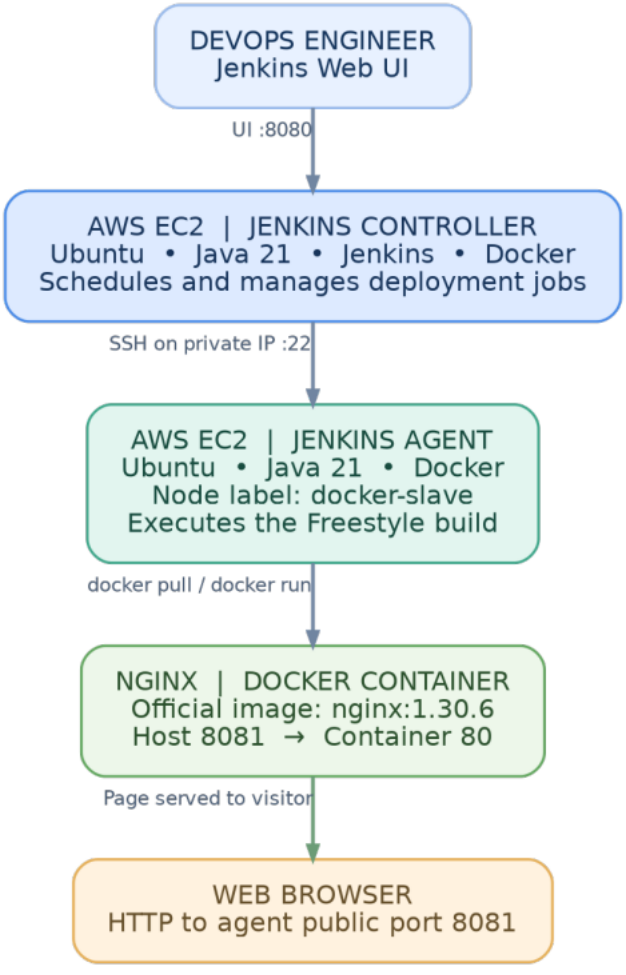
Topic	Official source
Jenkins Ubuntu/Linux installation	https://www.jenkins.io/doc/book/installing/linux/
Jenkins distributed agents	https://www.jenkins.io/doc/book/using/using-agents/
Jenkins SSH Build Agents plugin (`ssh-slaves`)	https://plugins.jenkins.io/ssh-slaves/
Docker Engine installation on Ubuntu	https://docs.docker.com/engine/install/ubuntu/
Docker Linux post-installation / group permissions	https://docs.docker.com/engine/install/linux-postinstall/
Docker CLI `docker run` / port publishing	https://docs.docker.com/reference/cli/docker/container/run/
Docker `pull`	https://docs.docker.com/reference/cli/docker/image/pull/
Docker `container inspect`	https://docs.docker.com/reference/cli/docker/container/inspect/
Docker `container rm`	https://docs.docker.com/reference/cli/docker/container/rm/
Docker Official Nginx image	https://hub.docker.com/_/nginx
Official Nginx supported tags (`1.30.6`, `stable`)	https://github.com/docker-library/official-images/blob/master/library/nginx
AWS security-group referencing/rules	https://docs.aws.amazon.com/vpc/latest/userguide/security-group-rules.html

Verification notes

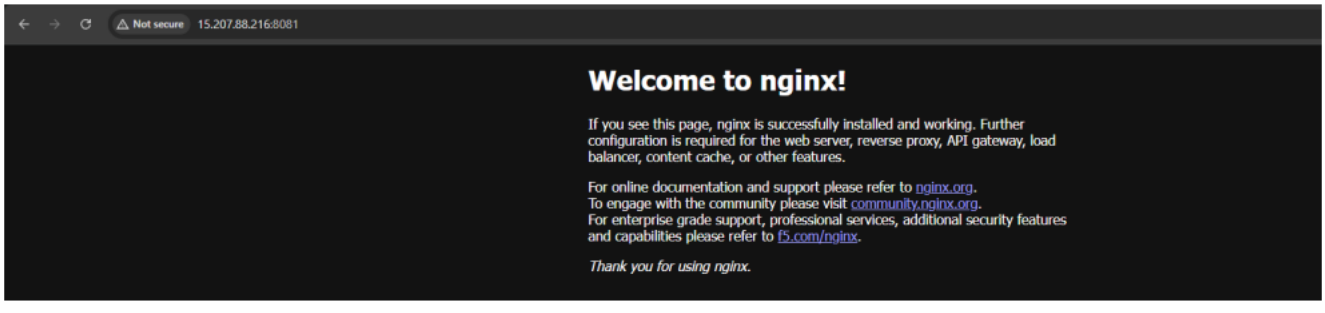
- The Docker Official Images manifest includes `1.30.6` and `stable` for Nginx's stable Debian image in its published tag list.
- The SSH Build Agents plugin's display name differs from its retained plugin ID, `ssh-slaves`.
- The AWS private-IP SSH path requires a valid private network route and compatible SG rules; simply putting instances on EC2 is insufficient.
- A Docker group user can effectively control the host. This design is for a dedicated training agent, not an unrestricted shared worker.

Evidence Screenshots

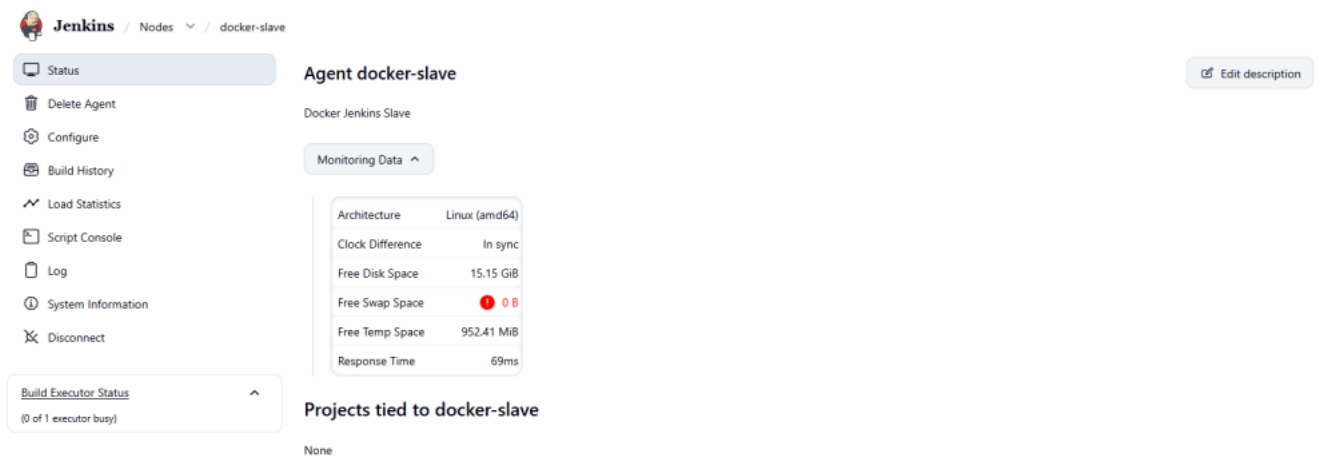
Jenkins deployment architecture and final Nginx website output.



Final Nginx website output



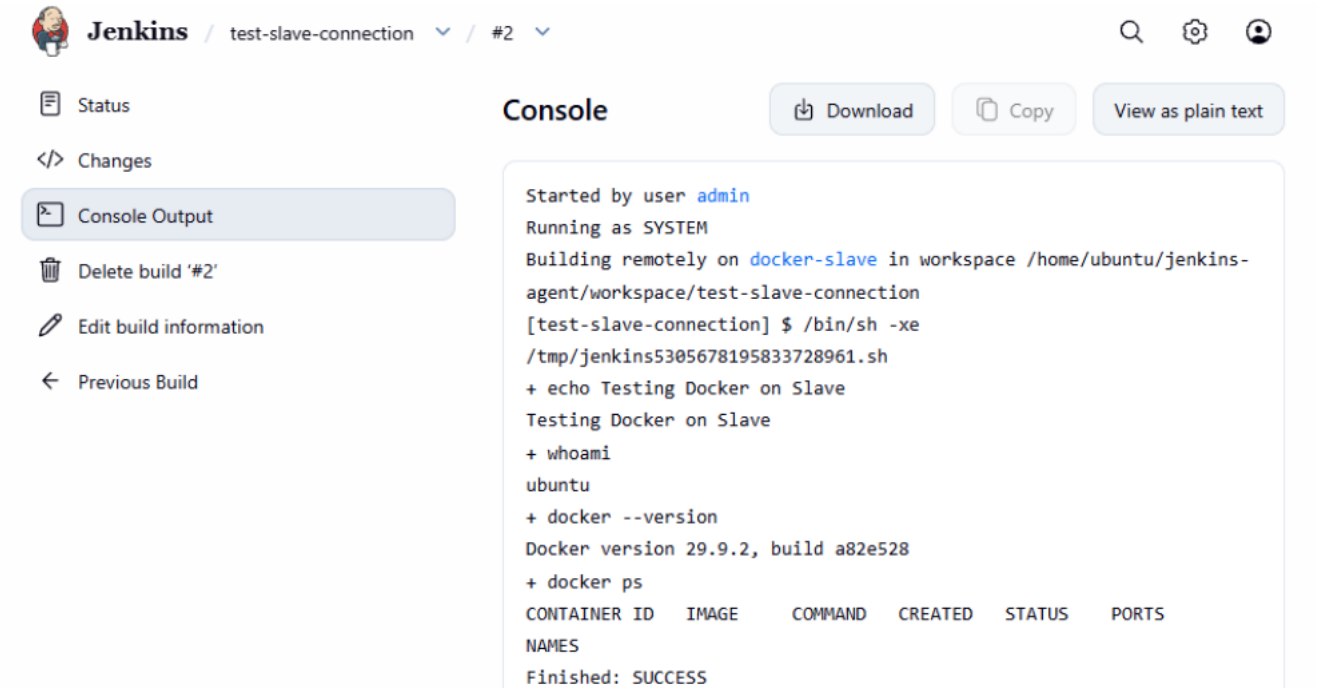
Nginx welcome page accessed through the agent on port 8081.



Agent online — observed



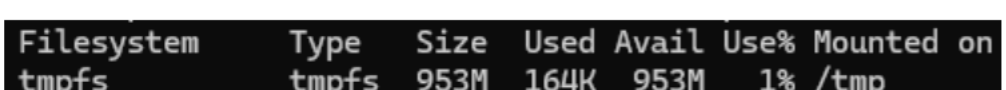
Remote job execution — observed



Docker access test — observed



Temporary-space warning — observed



tmpfs verification — observed